arXiv:2408.09536v3 [cs.SE] 28 Oct 2025

Galápagos: Automated N-Version Programming with LLMs

JAVIER RON, KTH Royal Institute of Technology, Sweden
DIOGO GASPAR, KTH Royal Institute of Technology, Sweden
JAVIER CABRERA-ARTEAGA, KTH Royal Institute of Technology, Sweden
BENOIT BAUDRY, Université de Montréal, Canada
MARTIN MONPERRUS, KTH Royal Institute of Technology, Sweden

N-Version Programming is a well-known methodology for developing fault-tolerant systems. It achieves fault detection and correction at runtime by adding diverse redundancy into programs, minimizing fault mode overlap between redundant program variants. In this work, we propose the automated generation of program variants using large language models. We design, develop and evaluate Galápagos: a tool for generating program variants using LLMs, validating their correctness and equivalence, and using them to assemble N-Version binaries. We evaluate Galápagos by creating N-Version components of real-world C code. Our original results show that Galápagos can produce program variants that are proven to be functionally equivalent, even when the variants are written in a different programming language. Our systematic diversity measurement indicates that functionally equivalent variants produced by Galápagos, are statically different after compilation, and present diverging internal behavior at runtime. We demonstrate that the variants produced by Galápagos can protect C code against real miscompilation bugs which affect the Clang compiler. Overall, our paper shows that producing N-Version software can be drastically automated by advanced usage of practical formal verification and generative language models.

CCS Concepts: • **Software and its engineering → Software development techniques**.

Additional Key Words and Phrases: N-Version Programming, Large Language Models.

1 Introduction

N-Version programming is a software engineering approach to reliability based on crafting software diversity [1] with different teams [2]. It is predominantly used to enhance the reliability of mission-critical systems [3]. This is achieved by building diverse and equivalent software components, with the objective that these would not fail all at once, under the same conditions, hence making the whole system more robust [3].

However, N-Version programming faces two fundamental challenges: First, empirical studies have shown that implementations developed by different teams might not be truly independent with respect to the target fault model, as developers tend to share similar mental models and processes, leading to correlated failures [4, 5]. Second, and the focus of this work, is the significant cost overhead of developing multiple versions. There are high costs induced by the manual development and verification of multiple program versions instead of a single one, typically linear in the number of versions [6]. Another important additional cost is the development of additional software to orchestrate the multiple versions (the N-Version harness). Our core insight is that LLMs can bring down the cost of engineering N-Version software.

Large Language Models (LLMs) are powerful tools able to carry out complex source code-related tasks, such as code completion [7], automatic program repair [8], and code summarization [9]. Our key intuition is that they are able to efficiently automate N-Version programming. Given a reference program, we can leverage the creative and

Authors' Contact Information: Javier Ron, javierro@kth.se, KTH Royal Institute of Technology, Stockholm, Sweden; Diogo Gaspar, dgaspar@kth.se, KTH Royal Institute of Technology, Stockholm, Sweden; Javier Cabrera-Arteaga, javierca@kth.se, KTH Royal Institute of Technology, Stockholm, Sweden; Benoit Baudry, benoit.baudry@umontreal.ca, Université de Montréal, Montreal, Canada; Martin Monperrus, monperrus@kth.se, KTH Royal Institute of Technology, Stockholm, Sweden.

```c
//main.c
int printf(const char *, ...);
static int a = -3, b;
static char c;
int d;
int e(int f, int g) {
  if (f - g < 10000)
    return f;
  return f + 1 % -f;
}
int main() {
  int *h[] = {&a, &a};
  for (; c <= 37; ++c) {
    int *i = &b;
    *i |= e(a, 8) + d;
  }
  printf("%d\n", b);
}
```

```
$ clang -v
clang version 17.0.0
Target: x86_64-unknown-linux-gnu

...

$ clang -O0 main.c; ./a.out
-3

$ clang -Os main.c; ./a.out
-1
```

Fig. 1. C code which triggers a miscompilation on clang v17.0.0, when using the -Os flag (left). After running the compiled binary, it prints -1. The correct result is -3 (right). Galápagos is able to detect and mitigate this miscompilation fault.

search power of LLMs to automatically produce program variants. We anticipate these variants to be diverse, due to the great volume of data used in LLM training, apt to being combined in an N-Version system. In other words, we want to harness the LLM's ability to synthesize program variants, and instead of having multiple teams producing multiple versions at a high costs, having the LLM producing them at virtually no cost.

In this paper, we design, implement and evaluate Galápagos, a tool for automated, verified N-Version programming. Galápagos performs three passes: incl. LLM prompting and program equivalence verification to provide strong guarantees. Galápagos generates variants of a reference function and then assembles N-Version implementations of that function. We implement Galápagos to harden C libraries, with support for producing same-language variants (C-to-C), and cross-language variants (C-to-Go).

We conduct a systematic evaluation of Galápagos to quantify the correctness, diversity, and usefulness of both the function variants and the N-Version components. First, we assemble a dataset of 30 real-world C functions, extracted from 6 notable cryptography and multimedia open source projects. We run Galápagos to transform each function in the dataset into an N-Version component. Second, we measure the diversity of the variants, both statically and dynamically. Statically, we measure the uniqueness of the resulting machine code after compiling the variants, using different optimization configurations. Dynamically, we execute each unique variant, and record the CPU instructions they utilize, comparing against state-of-the-art diversification and obfuscation tool Tigress [10] as a baseline. This allows us to quantify the variation in execution paths, which is known to be important against certain types of exploits such as side-channel attacks [11]. Third, we rigorously test the assembled N-Version implementations in the context of miscompilation bugs [12]. It clearly demonstrates Galápagos's ability to enhance robustness wrt. our target fault model.

Our experiments demonstrate several key findings: First, Galápagos uses LLMs to generate code variants that are provably equivalent, even when these variants are written in different programming languages. During our experiments, 234 *equivalent* variants were found from a dataset of 30 reference programs. Second, Galápagos is capable of producing and identifying code variants that exhibit diversity both in their code (statically) and during execution (dynamically), outperforming transformation-based diversification techniques like Tigress. From the dataset of 30 programs, 126 *unique* variants were verified as different, even after all compilation and optimization passes. Last but not least, these

diverse code variants can be utilized to strengthen critical sections of software via automated N-Version programming: we demonstrate Galápagos' capability to mitigate real-world, reported miscompilation bugs in the Clang compiler.

In summary, our contributions are

- The design and implementation of Galápagos, a tool for automated and verified N-Version programming using LLMs. As far as we are aware, this is the first work to realize an automatic N-Version programming framework with formal guarantees. Galápagos is a major contribution towards reducing the high costs of N-Version programming.
- A systematic suite of experiments to evaluate the correctness, value, and practicality of automated N-Version programming with Galápagos. The experiments consist of transforming real-world C code from notable open-source libraries into an N-Version counterpart.
- A publicly available, open-source repository for experimental reproduction and future research on automated N-Version programming, at https://github.com/ASSERT-KTH/Galapagos/

2 Background

In this section, we introduce the two areas in which our work is rooted: N-Version programming and neural machine translation.

2.1 N-Version Programming

N-Version programming is a software development approach, which consists of creating N implementations or *versions* of a specific program [2]. The core idea behind this approach is that when these versions are simultaneously executed, errors can be timely detected and mitigated by comparing their outputs. Ideally, the difference in implementations between versions is maximal, such that any coincidental errors are avoided. [13]. While originally devised as a fault-tolerance mechanism, N-Version programming has been adapted to enhance other specific properties of software, such as availability [14], reliability [15, 16], performance [17], or security [18, 19]

However, the enhancements offered by N-Version programming come with an attached trade-off, as it introduces many challenges throughout the software development lifecycle. These challenges include increased maintenance overhead, increased compute and memory use, or interoperability issues [14, 20]. Addressing these challenges requires additional effort and careful coordination across engineering teams.

An essential challenge is the increased cost of development, given that the time and resources needed to develop the versions increase at least linearly with N. To address this challenge, automating the process for creating new versions is a known and well-studied approach [11, 21–23]. In this paper, we contribute to this field of automatic synthesis of diverse program versions.

2.2 Code Translation with LLMs

Neural machine translation (NMT) for source code refers to the application of machine learning techniques for the automated translation between programming languages [24]. It aims to overcome the challenges associated with traditional rule-based or statistical translation methods by leveraging the power of neural networks to capture and model complex patterns inherent in source code [25]. The core principle behind NMT for source code involves training a neural network model on large datasets consisting of source code snippets in different programming languages and their corresponding translations [26]. During the training phase, the model learns to encode the source code syntax and

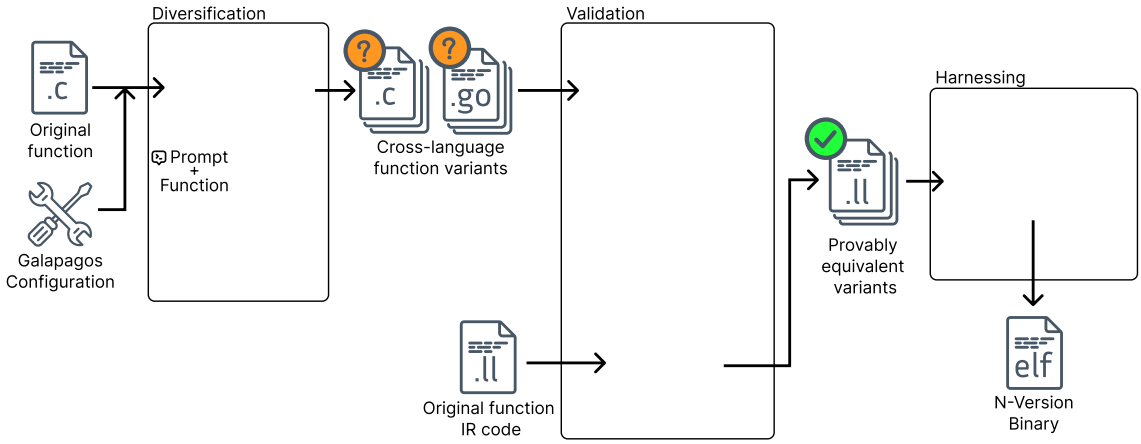


Fig. 2. Galápagos pipeline for automated, verified N-Version programming.

semantics into a continuous representation, enabling it to generate accurate translations. Subsequently, the trained model can be deployed to translate code snippets from one programming language to another automatically. NMT is tied to recent developments in large language models, which have proven to be efficient in performing translation tasks [27].

3 N-Version programming with LLMs

In this section, we present Galápagos, a tool that leverages large language models to harden software via automated N-Version programming. Galápagos works at the source-code level for generation, and at the intermediate representation (IR) level for verification. At the source-code level, it uses an LLM to produce new, diversified variants of program functions. At the intermediate representation level, these variants are verified to be semantically equivalent and to be finally assembled into N-Version functions. Because the variants are formally verified to be equivalent, Galápagos is able to provide strong guarantees about the behavior final N-Version assembly, irrespective of the introduced software diversity.

3.1 Fault Model

In this paper, we propose a novel technique to mitigate the class of faults called *miscompilation*. Miscompilations occur when a compiler erroneously generates machine code that deviates from the behavior defined in the source code, due to internal bugs in the compiler itself. These compiler faults are very hard to identify because they are typically silent. They are critical because the resulting application binaries behave incorrectly at runtime [28].

Miscompilations are present in all major compilers [29]. As of May 2025, Clang and GCC have 396 and 44 unresolved reports of miscompilation bugs, respectively. As an example, consider the code from an open LLVM bug report in Figure 1. After compilation using `clang` v17.0.0, with the `-Os` optimization flag, the result is a binary which produces incorrect output: `-1`. The compilation process exits "successfully" with code `0`, and offers no warnings or error messages which would hint that something has gone wrong. Hence, there is an error in one of Clang's optimization passes. This kind of fault can cause major downstream failures [30].

Let us abstract the compilation process as the following composition of functions:

$$P = B_a(M(F_l(S))) \tag{1}$$

In Equation 1, S is a program's source code; F_l is the front-end stage, which is specific to the language l, and transforms S to an intermediate representation (IR); M is the middle-end phase, i.e. a set of optimization passes performed on the IR; B_a is the back-end stage, which transforms the IR into assembly code targeted towards a specific architecture a; and P is the resulting executable program. Furthermore, M is subject to a wide range of configuration options, each of which can potentially introduce miscompilation bugs. For example, optimization flags such as `-Os` or `-O3` enable aggressive transformations intended to reduce code size or improve performance, but they also increase the risk of activating miscompilation bugs in these optimization passes.

In this work, we want to harden programs generated by the compilers' middle-end M and back-end B_a stages, i.e. any operations performed on the intermediate representation which result in P not behaving as specified by S.

3.2 Overview

Galápagos is a three-pass pipeline, as illustrated in Figure 2: *Diversification*, *Validation*, and *Harnessing*.

The pipeline's input is a function's source code, which is part of a larger application or library. For instance, a function could compute a cryptographic hash. Hereafter, we refer to this function as the *reference function* or simply the *reference*. The reference defines the expected functionality of the function on the whole input domain. In other words, the reference formally specifies the input/output behavior that all variants should conform to.

To create an N-Version implementation of the reference function, it is first processed in the Diversification pass (subsection 3.3) of Galápagos. Here, an LLM is prompted to automatically generate different *variants* of the reference. Second, the collection of generated variants is passed to the Validation pass (subsection 3.4), which filters out non-viable variants produced by the Diversification pass. The Validation pass proceeds through a sequence of validation steps: (1) Galápagos compiles each variant and produces its corresponding IR code, variants that cannot be compiled are filtered out as non-viable; (2) The IR code of all variants that compile is tested, and sent to a formal equivalence checker to compare their functionality against the reference. The variants that pass the tests and the equivalence check are then forwarded to the next pass of Galápagos. This ensures that after this pass, the ensuing variants have formal guarantees that the original functionality is preserved, and expected diverse internal behavior per the LLM instructions. Third, the Harnessing pass (subsection 3.5) uses the resulting variants to assemble an executable, where the original function is replaced by an N-Version implementation of that function. The specific LLM and equivalence checker are external and configurable, which allows Galápagos to use any off-the-shelf solutions.

Galápagos generates a binary that contains an N-Version implementation of the reference function. This binary protects against miscompilation faults, thanks to two properties: (1) miscompilations are very sensitive to the input source code, e.g. they might be related to a specific keyword [31]; consequently, deriving program variants from diverse-but-equivalent source code is less likely to trigger miscompilations in every diversified instance, and; (2) miscompilations are compiler-specific [12], meaning that it is improbable that variants produced by different compilers (see subsection 3.3) share the same miscompilation error.

3.3 Diversification Pass

The goal of this step is to systematically create function variants given a reference function.

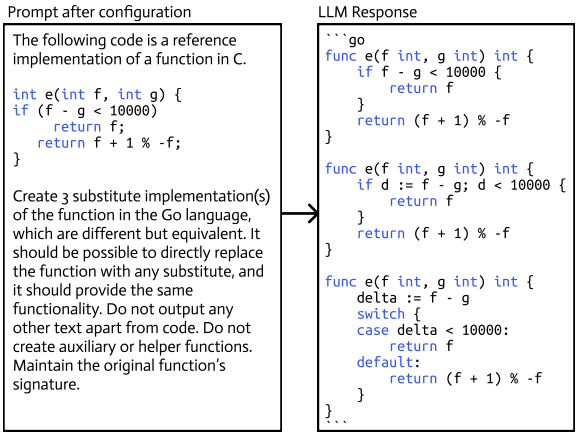


Fig. 3. Galápagos' diversification pass example. On the left box, the Galápagos prompt; on the right box, the response from the LLM.

As shown in Figure 2, the Diversification pass is managed by a controller module. The controller is responsible for reading configuration, interfacing with LLM APIs, and processing responses. The invocation of the LLM is configured by the following parameters: the LLM API where the diversification request will be sent to; the reference function's source code; the number of variants to be generated; and a pair of input and output languages. The input language is the one of the reference function. The output language can be the same one as the reference function's (e.g. diversifying C-to-C), or a different language (e.g. diversifying C-to-Go).

In Galápagos, the LLM prompt structure is as follows:

- A natural language description of the reference function: *The following code is a reference implementation of a function in <INPUT>.*
- The reference function.
- A description of the task: *Create <NUMBER> substitute implementation(s) of the function [in the <OUTPUT> language], which are different but equivalent. It should be possible to directly replace the function with any substitute, and it should provide the same functionality.*
- Additional remarks: *Do not output any other text apart from code. Do not create auxiliary or helper functions. Maintain the original function's signature.*

The *<NUMBER>*, *<INPUT>*, and *<OUTPUT>* strings are replaced according to the configuration, and the text in square brackets is omitted if the input and output languages are the same. Note that some LLM APIs provide support for sampling n times with the same prompt (aka n_sample). Our pilot experiment showed that a single prompt with n_sample > 1 produced less diverse results than one single prompts requesting for *<NUMBER>* variant. Hence, this is what we use in this paper.

Figure 3 shows an example of the diversification pass, including the resulting prompt after configuration, and resulting function variants. The configuration is set to generate three Go variants from a C reference function. Recall that translating to another language such as Go, provides runtime diversity which is the core goal of N-Version programming [2].

In the response, we can observe that while the first variant is a direct translation of the reference function, the second and third variants are dissimilar, using extra operations or different control flow constructs.

As shown in the previous example, the diversification pass can be performed in a cross-language configuration. This is possible because the Validation and Harnessing passes of Galápagos are performed at the IR level. Hence, the source languages for the reference and the variants can be different, as long as there exist front ends that can compile the languages to the same IR.

This diversification approach presents two main challenges. First, LLM outputs are not always correct [32]. We need a mechanism to guarantee that the produced variants are equivalent. This concern is addressed in the Validation pass (subsection 3.4) Second, when diversifying the reference to different programming languages, the IR code resulting from compiling the variants might not be compatible. We mitigate this concern in the Harnessing pass (subsection 3.5).

To sum up, the Diversification pass of Galápagos crafts an LLM prompt to produce variants of the reference function either in the same language or in a different language that can be transformed into the same IR. The resulting variants are forwarded to Galápagos' next pass.

3.4 Validation Pass

The goal of the Validation pass is to guarantee that the functionality of the function variants produced by the Diversification pass is equivalent to the reference.

In contrast to previous work where automatic diversification is achieved by performing transformations (1) that operate on low-level code representations, e.g. binary code, and (2) that are known to preserve equivalence [11, 33, 34], Galápagos leverages LLMs at the source code-level, and thus the variants are not guaranteed to be correct by construction [35, 36]. To address this issue, Galápagos performs four checks on each variant: compilation in isolation, compilation within containing project, test suite success, and equivalence check.

Compilation success in isolation: In this check, Galápagos inserts the variant into a file which only contains an entry point, e.g., a main function, that calls the variant with arbitrary parameters. Then we attempt to compile this file with the corresponding language's toolchain. The variant is passed to the next step if compilation completes without any error.

Compilation success within the corresponding project: In this check, Galápagos replaces the reference function's IR inside the global project with the variant's IR. The variant is filtered out if the build pipeline is unsuccessful. Reasons for failure at this step are related to IR incompatibility, e.g. mismatching types.

Test suite success: In this check, Galápagos executes the test suite against each binary that includes a valid variant created in the previous step. The variant is filtered out if any test from the suite does not pass. At this step, a test case that fails is direct evidence of a functionality difference between the original and the variant.

Equivalence check: In this final check, Galápagos calls a configurable off-the-shelf formal equivalence checking tool, such as alive-tv [37], or Rust's Kani [38]. This uncouples Galápagos from any specific IR language and allows the use of any state-of-the-art solution for verification. It performs the equivalence check with the reference and variant IR codes as parameters. The variant is filtered out if the tool proves the variant to be non-equal, or if it fails to prove equivalence within a given time and memory budget. Reasons for failure at this step are direct evidence of non-equivalence, as the variant deviates from the specification's functionality. This is the most resource-consuming check, so it is executed last. All function variants that pass the equivalence check are considered correct and can be forwarded to the next pass for assembly. Addressing the accuracy of the equivalence checking tool is out of the scope of this work. We point that equivalence check false negatives would not introduce errors to the system, as the variant would be filtered out. Equivalence check false positives are more challenging, as non-equivalent variants would be accepted in the N-Version

assembly, yet, their divergent output would not propagate silent corruptions, but instead would trigger a program termination (see subsection 3.5).

To sum up, the Validation pass filters candidate variants, eventually keeping only the variants for which it can provide formal correctness guarantees. Galápagos blends the radiating diversity powered by the LLM's creativity with strict guarantees of formal program equivalence checking.

3.5 Harnessing Pass

Per N-Version programming, the function variants need to be assembled into a single unit before execution. In Galápagos, the assembling is done at the intermediate representation level.

The previous passes of Galápagos aim at maximizing diversity. They leverage the diversity of source languages and blend multiple toolchains. Therefore, the variants' IR contain subtle differences, which is at the core of the software diversity we are seeking. Yet, these differences need to be dealt with when assembling N-Version functions. To handle this case, Galápagos supports non-trivial IR transformations of the variants during the harnessing pass. Specifically, Galápagos implements Go-to-C transformations related to function signatures and replacing language-specific components: (1) the LLVM function's `nest` self-reference is removed; (2) the function's parameter names are normalized; and (3) IR calls to the Go "runtime" module are removed [1].

Figure 4 shows an example where two equivalent variants of a function which adds two integers are harnessed into an N-Version function. This example is written in the LLVM intermediate representation. The variants' code is shown in the boxes on the left-hand side, and the resulting N-Version function is shown in the box on the right-hand side. In the N-Version form, the code from the single variants is inserted in the corresponding file as a function definition. These definitions are named after the original function plus a prefix. Galápagos then generates a wrapper function named after the original reference function. This function invokes all the different variants and compares their outputs. Galápagos also transforms the original application to redirect all existing invocations of the reference to the wrapper function. Finally, the binary is produced from the resulting IR code.

For N-Version programs, the output from the N functions can be selected in different ways. In Galápagos, we support N-of-N selection [39], i.e. we compare the return values of each variant against each other and return a value only if all the variants agree, otherwise Galápagos forcefully terminates the execution.

3.6 Class of Functions Considered for Automated N-Version

Galápagos transforms functions into N-Version functions, with equivalence guarantees. To achieve this challenging task, we must assume a specific class of functions. Namely, these functions must be pure, i.e. must deterministically produce the same output from the same input, and must not cause any side-effect. This is a requisite to our N-Version approach since all function versions must be executed simultaneously, and their outputs compared. Only with pure functions can we ensure that multiple executions of the function will not result in the accumulation of undesired side effects.

This scope for Galápagos is relevant, as pure functions represent a significant subset of all functions in software [40, 41]. Figure 5 shows a relevant example from the cryptography library libsodium that can be divsersified. Furthermore, we argue that in practice, critical sections of code that have strong reliability requirements can be refactored into pure functions, making hardening via Galápagos and N-Version programming possible.

[1] https://github.com/ASSERT-KTH/Galapagos/blob/main/linker/lib/go/go.cpp

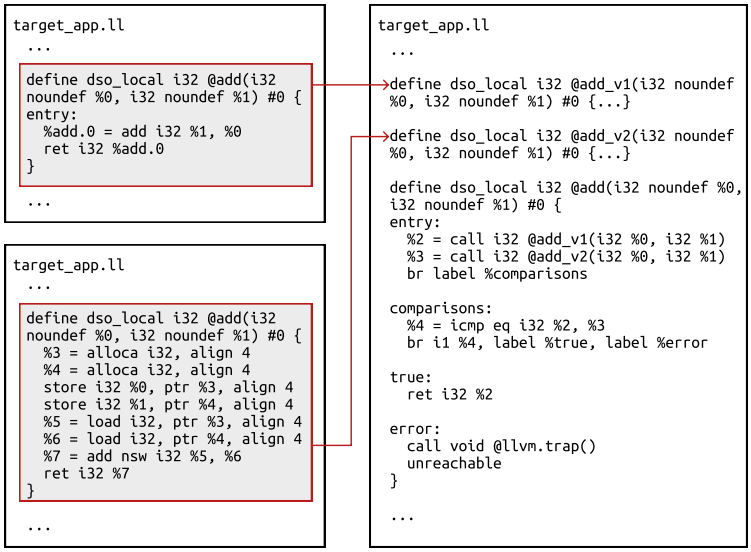


Fig. 4. Galápagos's harnessing pass example, in LLVM IR.

```c
static int
_crypto_scalarmult_ed25519_is_inf(const unsigned char s[32])
{
    unsigned char c;
    unsigned int  i;

    c = s[0] ^ 0x01;
    for (i = 1; i < 31; i++) {
        c |= s[i];
    }
    c |= s[31] & 0x7f;

    return ((((unsigned int) c) - 1U) >> 8) & 1;
}

```

Fig. 5. Pure function example from the cryptography library libsodium, which meets the criteria for automated N-Version programming with Galápagos.

3.7 Implementation

We implement the Diversification and Validation passes of Galápagos in Python.

The LLM used in our experiments is OpenAI's GPT-4o, because it was the best at the time of doing the experiments of this paper. The pipeline would support plugging other LLMs. It is left to future work to study the impact of varying the LLM on the diversification pipeline.

For formal equivalence checks, we rely on the `alive-tv` [37] tool that checks equivalence at the LLVM IR level. `alive-tv` is an equivalence checker tied to the LLVM IR and its semantics. It operates by comparing two versions of a function: source and target. These versions must be well-formed LLVM IR and adhere to the same LLVM specification version. It verifies that the target function preserves the semantics of the source function, and enforces that the target function must be a refinement of the source function. This is, for all inputs where the source function has defined behavior, the target must produce the same observable result. The comparison is made by translating both variants to

SMT and asks for logical equivalence. If the problem is satisfiable, it means that the solver has found a counter-example, that is an input such that both variants behave differently.

The Harnessing pass is implemented in C++ in order to make use of LLVM's code manipulation libraries. It programmatically combines the LLVM IR version of the variants and calls the N-Version output comparison function.

Galápagos supports C as input language, and C and Go as output languages. Support can be extended to any language pair, as long as these can be compiled to a common intermediate representation.

To provide maintanability and debugging in Galápagos, all outputs from each pass are saved to disk. This includes raw data from the LLM call, all variants' source code and all resulting IR and machine code.

The implementation of Galápagos can be found in our open source repository at https://github.com/ASSERT-KTH/Galapagos.

4 Experimental Methodology

In this section, we introduce the research questions that structure our empirical validation of Galápagos.

4.1 Research Questions

To systematically evaluate the effectiveness of Galápagos, we define the following research questions:

RQ1 *To what extent is a large language model able to produce functionally equivalent variants of programs at the source-code level?*

The variants generated by an LLM come with no guarantee. We aim to compute what percentage of these variants can be proven to perform the same task as the original function under diversification. To answer this question, we take a representative sample of functions from relevant C applications and libraries, and run them through Galápagos' pipeline. Then, we analyze the distribution of these variants in terms of the different correctness filters of Galápagos' validation passes presented in subsection 3.4.

RQ2 *To what extent do the program variants produced by a large language model exhibit diverse observable internal behaviors?*

The variants generated by an LLM can be different at the source code or IR code. However, program variants stored on disk do not increase reliability per se. N-Version design increases reliability when variants exhibit observable differences. For this, we compute how much of the variants' diversity is preserved after all compilation and optimization stages. Furthermore, we compare how much CPU instructions vary between different function variants.

RQ3 *To what extent are equivalent variants produced by a large language model useful to harden programs against the considered fault model?*

We evaluate the effectiveness of N-Version functions per our fault model, miscompilation bugs (see subsection 3.1). We curate publicly documented, reproducible miscompilation bugs, and determine whether LLM-generated variants are useful in detecting and preventing them.

RQ4 *What is the performance overhead introduced by the N-Version functions generated by Galápagos?*

Project	Version	# Commits	# Files	# Functions
alsa-lib	v1.2.11*	4 445	400	4 004
FFmpeg	n6.1-dev	111 662	4 160	110 532
libgcrypt	1.10-base*	3 419	588	5 616
liboqs	0.10.0*	1 458	3 625	9 594
libsodium	1.0.18	4 204	349	3 683
openssl	3.0.0-beta2	32 959	4 004	94 984

Table 1. List of real-world C projects used for experimental validation,. The versions marked with (*) represent the oldest named version before the commit used in this dataset.

N-Version functions with high execution overhead might be impractical. Therefore, we measure the resulting performance overhead introduced by Galápagos. To do this, we run the original and N-Version functions and compare their average execution times.

4.2 Dataset of Functions

We select C functions from six open-source projects: alsa-lib, FFmpeg, libgcrypt, liboqs, libsodium, openssl. Those projects are all real-world, and have differing sizes, stages of maturity, and adoption. Table 1 shows the selected projects, their versions, and related statistics. The number of commits for the selected projects' repositories ranges between 1 458 and 111 662. Likewise, the number of C files and declared functions goes from 349 to 4 160, and 3 683 to 110 532 respectively.

From each of these projects, we select the first five functions that meet the following properties, analyzed sequentially. (1) It is a pure function, and; (2) It does not call other functions. These properties ensure that the functions are supported by the semantic equivalence tool used in Galápagos' verification step.

Table 2 describes the set of functions we selected, which covers a diverse range of functionalities, e.g. arithmetic, type conversion, and cryptographic operations. It also shows each function's size in lines of code (LOC), and the number of times that it is statically referenced within the project (#Refs). The size of the functions by lines of code ranges from 2 to 34 with a median of 10. The number of times that each function is called within the same project ranges from 1 to 1 220 with a median of 2. To sum up, this dataset is exclusively composed of real-world C code that is used daily by millions of users.

4.3 Methodology for RQ1: Assessing Functions Diversified by LLMs

As part of its Diversification pass, Galápagos generates an arbitrary number of raw variants. However, there are mandatory properties which every variant needs to comply with for N-Version design: compilability, test suite execution success, and formal equivalence assessment. To have a detailed view of the distribution of equivalent and non-equivalent variants, we measure the proportion of variants which fail at each filter and the reasons behind the failures. Ultimately, the effectiveness of LLM-based diversification is measured by the proportion of variants which pass all filters and are proven equivalent.

Importantly, we diversify the dataset of C functions (subsection 4.2) with two configurations same-language (C-to-C) and cross-language (C-to-Go). We produce 10 variants for each configuration, i.e. 100 variants per project, and 600 variants in total. Each variant is checked against all the filters described above, and the pass/fail occurrences are reported.

Project	Function Name	Size (LOC)	# Refs.
alsa-lib	alaw_to_s16	15	1
	iec958_parity	13	1
	add	6	3
	ulaw_to_s16	15	1
	val_seg	15	2
FFmpeg	flac_get_max_frame_size	19	2
	mix	6	43
	vp5_adjust*	14	2
	weight	10	2
	int_sin	17	5
libgcrypt	barrett_reduce	7	2
	ctz	12	1
	int16_t_negative_mask	7	2
	int16_t_nonzero_mask	7	5
	montgomery_reduce	7	2
liboqs	fpr_half	11	132
	fpr_lt	29	80
	modp_montymul	9	1 220
	modp_norm	2	80
	int16_nonzero_mask	6	16
libsodium	b64_byte_to_char	6	4
	b64_byte_to_urlsafe_char	6	4
	b64_char_to_byte	9	2
	b64_urlsafe_char_to_byte	9	2
	fBlaMka	5	28
openssl	icbrt64	15	1
	BitDeinterleave	34	1
	BitInterleave	34	1
	_booth_recode_w5	10	3
	_booth_recode_w7	10	2

Table 2. List of 30 candidate real-world C functions for diversification. LOC "stands for lines of code". # Refs. stands for number of times that the function is statically referenced within the project.

Tag	Version	Description	Issue #	Status
M1	15.0.0	Wrong code with `-O1`	58765	Unresolved
M2	17.0.0	Wrong code with `-Os`	68871	Unresolved
M3	18.0.0	Wrong code with `-march=znver4 -flto -O3`	80494	Unresolved

Table 3. List of known miscompilation bugs selected for mitigation through Galápagos.

4.4 Methodology for RQ2: Assessing Variants Uniqueness

To answer RQ2, we assess the diversity of the variants both statically and dynamically.

Diversity at source code level can be removed through the transformations performed at different stages of compilation [42]. We measure how often this occurs, collecting the proportion of variants that preserve uniqueness after compilation with different optimization flags. We compare the unique SHA-256 hashes of the produced machine code, assuming that compilation is repeatable. This experiment is performed on the same dataset of C functions as in RQ1 (4.2). We perform this comparison only for variants that have been proven functionally equivalent previously by the filters applied in 4.3. We use each of four optimization flags `-O0`, `-O1`, `-O2`, `-O3` to rebuild the function's machine code, and compare the SHA256 hash of the produced binaries, to inspect whether some compilation flags undo specific variants (i.e. make the binary statically identical to the original).

Then, we execute all statically diverse variants, to measure the diversity of observable behaviors among them. We define diverse observable behaviors per the CPU instruction trace. At runtime, we record all CPU instructions executed by the variant. To achieve this, we rely on Intel's Pin instrumentation tool [43], since it allows us to record CPU instructions at function granularity, filtering out all other execution inherent to the used runtime, e.g. state initialization, garbage collection, etc. We then aggregate and compare the instructions executed by each variant in two ways. First, we compare the CPU instruction set used by the variants and compare it against the set of the original CPU instruction set. This is measured using the Jaccard similarity coefficient [44]. Second, we compare the amount of CPU instructions executed by the variants in terms of absolute numbers.

Finally, we compare the dynamic diversity of the LLM-generated variants against a baseline of rule-based variants generated with state-of-the-art tool Tigress [10]. Tigress is a code transformation tool developed and maintained by Collberg and colleagues [45] for diversification and obfuscation. We consider Tigress a suitable baseline approach since: (1) it works at the same level of abstraction; and (2) it aims to achieve behavioral diversity through program transformation, aligning with the objectives of N-Version programming. We apply the control-flow flattening transformation and use the resulting variants for comparison at runtime using the Jaccard similarity coefficient.

4.5 Methodology for RQ3: Validating the Ability to Mitigate Miscompilations

As described in subsection 3.1, our work specifically focuses on hardening against miscompilation errors produced by bugs in compilers, which is a very dangerous and tricky class of bugs. In RQ3, we consider miscompilation bugs from the Clang compiler. We assess the effectiveness of Galápagos in automatically generating N-Version programs which can mitigate these bugs. We look at all bugs reported in Table 3, coming from different versions of Clang and optimization stages. All are unresolved in the repository's issue tracker at the time of experimentation. For each bug, we create two test programs: one with a function containing a minimal reproducible example for said bug, and one with an N-Version implementation of the same function. We always run Galápagos with the minimal function as the reference broken run. We consider Galápagos to be successful if the N-Version implementation of the reference forces a crash instead of producing the incorrect output due to the miscompilation bug.

4.6 Methodology for RQ4: Measuring Performance Overhead

To measure the performance overhead of N-Version functions generated by Galápagos, we use the `clock_gettime(3)` API. To get an accurate comparison, we time both the original single-version and the N-Version functions over 2 000 000 executions. Execution and measurements are done in an Intel i7-11800H system, with 16 GB of RAM.

```
add.c
 static inline unsigned int add(...){
     if (__builtin_add_overflow(a, b, &a))
         return 4294967295;
     return a;
 }
```

Fig. 6. Build failure example: Function '`__builtin_add_overflow`' is not declared in the original project's IR.

5 Experimental Results

In this section, we present and discuss the results of our experiments. In subsection 5.1 we look at the ability of GALÁPAGOS to generate variants that are equivalent to the reference (RQ1). In subsection 5.2 and subsection 5.3, we assess the static and dynamic diversity of the variants, to answer RQ2. We conclude with subsection 5.4, where we answer RQ3 with empirical evidence about the resulting N-Version units' ability to mitigate miscompilation errors.

5.1 Validating Variants

To answer RQ1, we analyze the extent to which the generated variants pass the different validation filters. The numbers are presented in Table 4.

First, we discuss the results for the generation of variants, in the same language. On average, 99,3% (298/300) of the variants are compiled correctly in isolation, and, when inserted in the project, 96,0% (288/300) still successfully compile. Compilation in isolation fails in 2 out of 300 cases: one case because of a syntax error, and the second because the variant calls a function from a library that has not been imported. Compilation within the project fails in 10 out of 298 remaining cases, all of them because the variant's IR code makes use of a function that is not declared in the project's IR code. For instance, one of the variants of alsa-lib's `add` uses a compiler built-in function, and compiles in isolation, shown in Figure 6. Yet, it fails to compile inside the project, as the built-in function is not declared in the IR scope of the reference function.

The average rate of success decreases for test validation and equivalence verification to 82,6% (248/300) and 34,3% (102/300), respectively. Testing fails in 40 out of 288 remaining cases, either because of at least one failed test case or test suite timeout. An example from ffmpeg's `flac_get_max_frame_size` is shown in 7, where constant addition to the `count` is not performed correctly. Finally, equivalence validation fails in 151 out of 248 remaining cases. Here, we uncover the cases where the LLM is able to synthesize variants that successfully run and behave similarly to the original, yet the complete semantics of the reference function are not correctly captured. Figure 8 shows an equivalence validation failure example for one of the variants of function `fBlaMka`. The figure shows a snippet of `alive-tv`'s that indicates that for inputs `x` and `y` of 131 074 and 51 111 936 respectively, the reference function outputs 13 398 943 041 538, while the variant outputs 7 235 012 610.

For the cross-language configuration, on average, 92,6% (278/300) of the variants are compiled correctly in isolation, yet, when inserted in the project, only 76,3% (229/300) result in compilation success. Compilation in isolation fails in 22 out of 300 cases, because of syntax errors and failure of compile-time checks, e.g. overflow checks. Figure 9 shows an example where the code fails to compile, as it fails to pass the go compiler's integer overflow check.

Compilation within the project fails in 49 out of 278 remaining cases, with the reasons behind these failures being (1) the use of IR components in the variant which are not declared in the project's IR code, and; (2) IR type mismatches. At this stage, an interesting case is openssl's `_booth_recode_w7` function. This function fails to compile within the project

Project	Function Name	Same-language diversification (C)				Cross-language diversification (C to Go)			
		Comp. Isol.	Comp. Proj.	Test	Verif.	Comp. Isol.	Comp. Proj.	Test.	Verif.
alsa-lib	alaw_to_s16	10	10	10	1	10	10	10	6
	iec958_parity	10	8	8	8	10	10	10	10
	add	10	9	9	9	10	1	1	1
	ulaw_to_s16	10	10	10	9	10	10	10	10
	val_seg	10	10	10	2	9	1	1	0
ffmpeg	flac_get_max_frame_size	10	10	9	4	10	10	10	0
	mix	10	10	10	6	10	10	10	0
	vp5_adjust	10	10	10	0	6	0	0	0
	weight	10	10	10	10	10	10	10	0
	int_sin	10	10	10	0	10	10	10	0
libgcrypt	barrett_reduce	10	10	0	0	10	10	10	10
	ctz	10	10	10	9	10	10	10	6
	int16_t_negative_mask	9	9	7	7	10	8	6	5
	int16_t_nonzero_mask	10	10	7	7	9	9	5	5
	montgomery_reduce	10	10	6	4	10	10	8	8
liboqs	fpr_half	10	10	10	2	10	10	10	1
	fpr_lt	9	9	9	0	9	9	9	1
	modp_montymul	10	10	10	2	10	10	9	0
	modp_norm	10	10	7	0	8	8	3	2
	int16_nonzero_mask	10	10	10	3	10	10	10	8
libsodium	b64_byte_to_char	10	8	8	0	10	6	6	0
	b64_byte_to_urlsafe_char	10	8	8	0	10	9	9	0
	b64_char_to_byte	10	7	0	0	9	8	0	0
	b64_urlsafe_char_to_byte	10	10	0	0	8	0	0	0
	fBlaMka	10	10	10	9	10	10	10	0
openssl	icbrt64	10	10	10	7	9	9	9	0
	BitDeinterleave	10	10	10	0	10	10	10	3
	BitInterleave	10	10	10	1	1	1	1	1
	_booth_recode_w5	10	10	10	2	10	10	10	2
	_booth_recode_w7	10	10	10	0	10	0	0	0
avg.		99.3%	96.0%	82.6%	34.0%	92.6%	76.3%	69.0%	24.3%

Table 4. Count of variant compliance to each validation filter: Compiled in isolation (Comp. Isol.), compiled within project (Comp. Proj.), test suite success (Test), and verified equal (Verif.).

because the variants have a modified signature: The LLM added a suffix for each variant, disregarding the prompt's explicit instructions to keep the original signature.

When executing the variants, the average rate of success drops for test validation and equivalence verification to 69,0% (207/300) and 24,3% (73/300), respectively. Testing fails in 22 out of 229 remaining cases, either because of at least one failed test case or test suite timeout. The acquired errors are logical ones, such as the one displayed in Figure 7, with no major LLM hallucinations detected in the variants generated in our experiments. This is, no nonsensical, or

flac_get_max_frame_size.c

```
static int flac_get_max_frame_size(...){
    int count;
    count = 16;
    count += ch * ((7+bps+7)/8);
...
}
```

flac_get_max_frame_size.c

```
static int flac_get_max_frame_size(...){
    int count = 16;
    count += (ch * (bps + 14) + 7) / 8;
...
}
```

Fig. 7. A logical error in variant (bottom) which causes test failure. The addition to count is not equivalent.

```
Example:
i64 noundef %x =
#x0000000000020002 (131074)
i64 noundef %y =
#x00000000030be800 (51111936)
================================

Source:
...
i64 %add3 = #x00000c2faf3d8802
(13398943041538)

================================

Target:
...
i64 %add4 = #x00000001af3d8802
(7235012610)

================================

Source value: #x00000c2faf3d8802
(13398943041538)
Target value: #x00000001af3d8802
(7235012610)
```

Fig. 8. Formal equivalence validation failure: parameters x and y produce different outputs in the reference function and the variant.

completely off-the-mark solutions. Finally, equivalence validation fails in 110 out of 207 remaining cases, where the LLM misses a part of the semantics of the reference function.

It is worth noting that the acceptability of variants is close for both same-language and cross-language configurations, especially when contrasting the final number of variants that are proved to be equivalent.

Furthermore, for most functions (21/30), both configurations generate a similar amount of equivalent variants. Both same- and cross-language seem to excel or face difficulties for these functions: `libsodium`'s conversion functions are a clear example of this, consisting of a single line with several chained operations; they appear to *confuse* the LLM, regardless of the diversification configuration. The remaining functions (9/30) show disparate diversification results

modp_norm.go

```
func modp_norm(...) int32 {
    return int32(
        x - (p & uint32((int32(
                x-((p+1)>>1))>>31
            )^0xfffffffffffffffe
        ))
    )
}
```

Fig. 9. Build in isolation failure example. The code fails with an integer overflow error.

across configurations: while for some functions the same-language configuration performs better (5/9), for others the cross-language configuration produces more equivalent variants (4/9).

We now analyze the distribution of equivalent variants per function. In the same-language configuration, for the bottom quartile, 0 equivalent variant were found; for the median, 2 equivalent variants were found; and the top quartile, 7 equivalent variants were found. In the cross-language configuration, for the bottom quartile 0 equivalent variant were found; for the median 1, equivalent variant was found; and for the top quartile, 5 equivalent variants. We can observe that both distributions are skewed towards zero, this means that non-equivalent variants are more likely to be generated.

However, it is notable that for most (23/30) of the functions, at least 1 equivalent variant was generated. We consider this to be a promising result that shows the feasibility of the Galápagos concept. We believe this can be further improved by using more advanced models and sophisticated prompting techniques.

Answer to RQ1
Large language models successfully generate variants given a reference function. Out of 600 function variants, 170 were verifiably equivalent, 97 of which were created from same-language source code and 73 from a source code in a different language. This signals compelling applicability of creating automated cross-language N-Version functions with strong formal guarantees.

5.2 Static Uniqueness

Table 5 shows the number of variants which compile to LLVM-unique IR code, and produce a unique machine code with different optimization flags. The column "Total eq." shows the total number of equivalent variants found, as displayed in Table 4. The column "IR" shows the number of these variants that are unique IR files. We obtain this number by comparing the "Total eq." variants' hashes at the IR level. Subsequent columns, "-O0" to "-O3", show the number of unique machine code produced by the compiler with each optimization configuration.

For the same-language configuration, 92 of the 98 equivalent variants are different from each other at the IR level. These 92 variants are generated from 19 out of the 30 reference functions. The compilation with "-O0" preserves 81 unique variants. Meanwhile, the number of unique variants is reduced to 73 after "-O3" optimization, representing 74.48% of all equivalent variants. The optimizations observed in the functions are performed at the "-O1" level, and are concerned with inlining, loop invariant code motion, and global value numbering.

For the cross-language configuration, a total of 36 unique variants were found at the IR level, among the 73 equivalent variants.

Project	Function Name	Same-language diversification (C)						Cross-language diversification (C-to-Go)						Totals	
		Total eq.	IR	Binary				Total eq.	IR	Binary				IR	-O3
				-O0	-O1	-O2	-O3			-O0	-O1	-O2	-O3		
alsa-lib	alaw_to_s16	1	1	1	1	1	1	6	4	3	2	2	2	5	3
	iec958_parity	4	3	3	3	3	3	4	4	3	2	2	2	7	5
	add	9	9	9	9	9	9	1	1	1	1	1	1	10	10
	ulaw_to_s16	9	9	8	8	8	8	10	2	1	1	1	1	11	9
	val_seg	2	2	2	2	2	2	–	–	–	–	–	–	2	2
ffmpeg	flac_get_max_frame_size	4	4	3	3	3	3	–	–	–	–	–	–	4	3
	mix	6	6	5	5	5	5	–	–	–	–	–	–	6	5
	weight	10	9	8	8	8	8	–	–	–	–	–	–	9	8
libgcrypt	barrett_reduce	–	–	–	–	–	–	10	2	1	1	1	1	2	1
	ctz	9	9	9	9	9	9	6	5	4	4	4	4	14	13
	int16_t_negative_mask	7	5	5	1	1	1	5	3	2	1	1	1	8	2
	int16_t_nonzero_mask	7	7	6	4	4	4	5	4	1	1	1	1	11	5
	montgomery_reduce	4	4	4	4	4	4	8	1	1	1	1	1	5	5
liboqs	fpr_half	2	2	2	2	2	2	1	1	1	1	1	1	3	3
	fpr_lt	–	–	–	–	–	–	1	1	1	1	1	1	1	1
	modp_montymul	2	2	2	2	2	2	–	–	–	–	–	–	2	2
	modp_norm	–	–	–	–	–	–	2	1	1	1	1	1	1	1
	int16_nonzero_mask	3	3	2	1	1	1	8	3	1	1	1	1	6	2
libsodium	fBlaMka	9	9	6	6	6	6	–	–	–	–	–	–	9	6
openssl	icbrt64	7	5	3	2	2	2	–	–	–	–	–	–	5	2
	BitDeinterleave	–	–	–	–	–	–	3	2	2	2	2	2	2	2
	BitInterleave	1	1	1	1	1	1	1	1	1	1	1	1	2	2
	_booth_recode_w5	2	2	2	2	2	2	2	2	1	1	1	1	4	3
Total		98	92	81	73	73	73	73	36	24	21	21	21	128	94

Table 5. Variant uniqueness. Each column for both configurations shows how many unique variants are obtained at the IR level, and at the machine code level after various optimizations. Functions with no equivalent variants are omitted. The two rightmost columns display the total unique variants for both configurations, both at the IR level and at the machine code level, after building with the -O3 optimization flag.

These 31 variants correspond to 16 out of 30 reference functions. On compilation, the number of unique variants is reduced to 21 after "-O3" optimization, representing 28.76% of all equivalent variants.

A key result for RQ2 is the fact that the introduced diversity is generally well-preserved across optimization stages: from all 128 unique variants at the IR level, a total of 94 (73.43%) are still unique after applying all optimization passes.

In total, at least 1 unique, equivalent variant was found for 24 out of 30 reference functions. Furthermore, we are able to find at least one behaviorally unique variant for each function for which we successfully synthesize at least 1 equivalent variant. In the case of same-language diversification, the number of unique variants per function is greater than 2 in most cases (15/19). Cross-language diversification, on the other hand, does not perform as well, with only one unique variant for most functions (13/16). We hypothesize that the lower number of unique variants in the

cross-language configuration is caused by two reasons: The LLM used in our experiment produces less diverse source code given our experimental parameters, or; the Go-to-IR compiler produces more consistent code across variants.

5.3 Dynamic Uniqueness

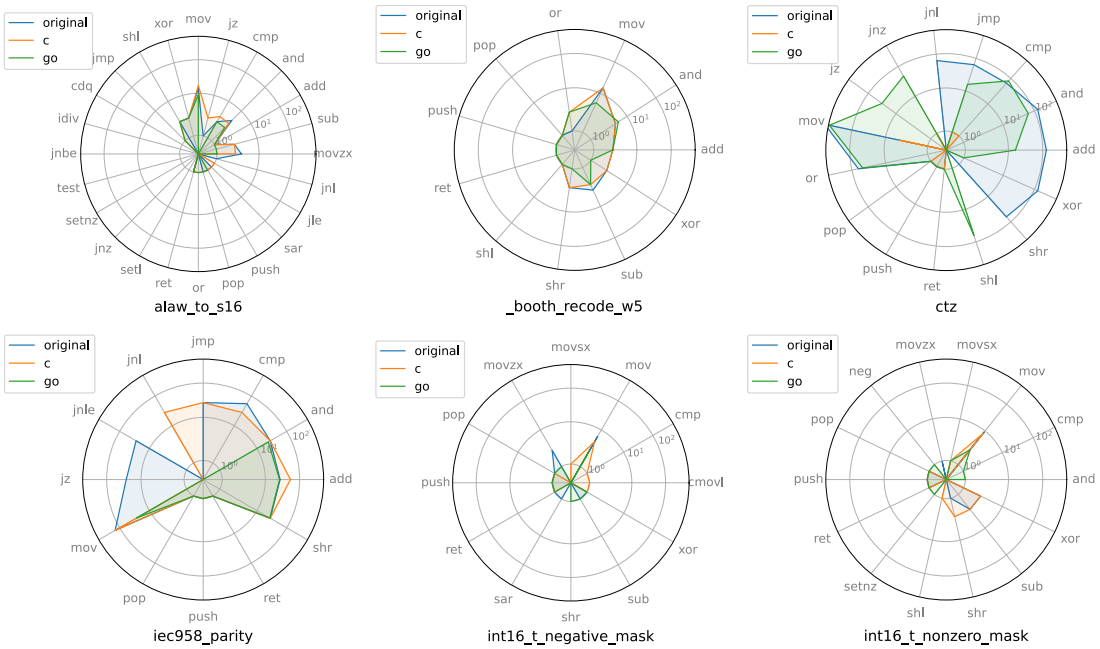


Fig. 10. Dynamic uniqueness of variants. Each radial plot shows the count of CPU instructions executed by the original function, superimposed with the instruction counts of the same-language (C) and cross-language (Go) variants.

Now, we execute the diverse, equivalent variants generated by Galápagos. Figure 10 shows plots with CPU instruction counts of 6 of the dataset's functions, one function per project. Each sub-figure shows 3 overlapping areas, aggregating the instruction counts of the original function, an equivalent same-language variant, and an equivalent cross-language variant. In detail, each figure is a radial plot, where the axes represent each of the CPU instructions executed by the function variants. Each colored area aggregates the instruction count of a different function implementation. This means that the less overlap in the figures, the more diverse the variants are in terms of the number and type of executed CPU instructions.

On these plots, we observe that the cross-language variants consistently show more diverse behavior in terms of CPU instructions. For instance, the sub-figure of function `_booth_recode_w5` shows an almost complete overlap of the original implementation and the same-language variant's areas, while the cross-language variant area clearly displays a different set of instructions and count of the overlapping instructions. Similarly, in the case of function `ctz`, the cross-language variant executes 3 instructions that are executed neither in the reference function nor in the same-language variant, while also avoiding 3 instructions from the original execution set. This difference in instruction subsets follows naturally from using different compiler front-ends for producing the variants' IR code.

Project	Function Name	C-to-C variants			C-to-Go variants			Tigress variant
		Min	Median	Max	Min	Median	Max	
alsa-lib	add	0.538	0.700	0.800	**0.462**	0.462	0.462	0.818
	iec958_parity	0.750	0.750	0.833	**0.500**	0.636	0.636	0.733
	val_seg	0.727	1.000	1.000	–	–	–	**0.600**
ffmpeg	flac_get_max_frame_size	1.000	1.000	1.000	–	–	–	**0.688**
	mix	1.000	1.000	1.000	–	–	–	**0.714**
libgcrypt	barrett_reduce	–	–	–	0.900	0.900	0.900	**0.769**
	ctz	0.462	0.462	0.643	**0.444**	0.538	0.667	0.750
	int16_t_negative_mask	**0.417**	0.417	0.700	0.556	0.778	0.778	0.727
	int16_t_nonzero_mask	0.357	0.455	0.700	**0.308**	0.308	0.308	0.727
	montgomery_reduce	0.800	0.800	0.900	**0.636**	0.636	0.636	0.750
liboqs	fpr_half	**0.727**	0.727	0.727	0.875	0.875	0.875	**0.727**
	fpr_lt	–	–	–	**0.412**	0.412	0.412	0.750
	int16_nonzero_mask	0.357	0.417	0.417	**0.286**	0.286	0.286	0.750
	modp_montymul	**0.750**	0.750	1.000	–	–	–	0.769
	modp_norm	–	–	–	1.000	1.000	1.000	**0.727**
libsodium	fBlaMka	0.875	1.000	1.000	–	–	–	**0.667**
openssl	BitDeinterleave	–	–	–	**0.364**	0.500	0.500	0.571
	BitInterleave	**0.333**	0.333	0.333	0.500	0.500	0.500	0.571
	_booth_recode_w5	1.000	1.000	1.000	1.000	1.000	1.000	**0.786**

Table 6. Jaccard coefficient comparison of executed x86 instructions to quantify runtime diversity introduced by Galápagos and Tigress' flatten transform. For each function, we list the minimum, median, and maximum Jaccard coefficients for the C-to-C and C-to-Go variants produced by Galápagos, as well as the coefficient for the variant generated by Tigress. Lower coefficients indicate less overlap with the reference implementation and thus greater runtime diversity. Bold values indicate the greatest diversity (lowest coefficient) observed for each function. Missing entries (–) correspond to configurations where no equivalent variant could be produced.

Finally, Table 6 reports the overlap between the variants and their reference implementations, measured with the Jaccard coefficient over executed x86 instructions. Across 12 of the 19 functions, at least one variant produced by Galápagos attains a lower Jaccard coefficient than the variant produced by Tigress' flatten transform. Overall, Galápagos's variants exhibit lower coefficients on average than Tigress', indicating less overlap and therefore greater runtime diversity. Moreover, the minimum observed Jaccard coefficient for Galápagos is 0.286, compared to 0.636 for Tigress' flatten transform. Lower coefficients imply smaller overlaps in executed instructions, i.e., greater diversity at runtime.

Focusing on Galápagos's variants, among the 11 functions where both same-language (C-to-C) and cross-language (C-to-Go) equivalents were found, 8 have their lowest coefficient coming from a cross-language variant. This suggests that cross-language variants are generally more effective at maximizing runtime diversity.

Answer to RQ2

Our experiments show that Galápagos is able to create function variants that are diverse to a large extent, as measured through their persistence after compilation and optimization. In total, for 166 unique variants across the

30 reference functions' source code, 126 (75.9%) remained unique after all compilation and optimization passes. This demonstrates that Galápagos's approach to diversity is robust to different optimization requirements.

We also compared Galápagos to a state-of-the-art rule-based diversification tool, Tigress. While Tigress is able to generate variants with runtime diversity, our results show that Galápagos generally achieves greater runtime diversity in the executed x86 instruction sequences. For 12/19 functions, the variants produced by Galápagos exhibit lower Jaccard coefficients relative to the reference implementation than those produced by Tigress, indicating less overlap and thus greater runtime diversity. This suggests that Galápagos's translation-based approach can introduce a higher degree of behavioral diversity than traditional rule-based diversification transformations.

In general for the generated variants, we observe diverse internal behavior at runtime, evidenced by different sequences of CPU instructions. This finding is relevant, as it means that the introduced diversity at the machine code level goes beyond minor changes in the binary. It is aligned with the core assumption of N-Version programming, that the fault-tolerance increases with runtime diversity.

5.4 Mitigating Miscompilations

Table 7 shows the results of mitigating the Clang miscompilation bugs introduced in Table 3 with Galápagos. First, we observe that Galápagos successfully generates multiple equivalent variants for each function that triggers each miscompilation bug. Second, Galápagos is able to assemble N-Version binaries for these functions, which caused the execution to crash, instead of producing a wrong output. We consider Galápagos effective in mitigating the three proposed bugs.

For bug M1, Galápagos produces 10 equivalent variants in the same-language configuration. It then assembles an 11-Version implementation of the function which triggers the miscompilation fault. The key result here is in the ability of the 11-Version function to let developers know that something wrong has happened. The original function compiles and, when it executes, produces a value. However, the value is wrong because of the miscompilation bug, but there is no warning or check to let the developers know about this wrong value. On the other hand, the 11-Version function compiles, and when it executes, it crashes because the results returned by each version are not consistent. In this case, the program does not silently introduce a wrong value and the developer can act upon this. Consider the control flow graphs in Figure 11. The graph on the left represents the execution of a single-version implementation of the M1 function. The miscompiled code could result in the program silently choosing the wrong execution path, with potentially dangerous results. The graph on the right represents the execution of an N-Version implementation of the M1 function. The miscompiled code executes and returns silently; however, if at least any of the other 10 versions returns a distinct value, the program will exit, thus avoiding wrong execution and informing the program operator that something wrong has occurred. This behavior is similar for the N-Version implementations that Galápagos produced for M2 and M3, with N values of 11 and 3, respectively.

Bug	Result	Configuration	#Eq. variants #
M1	Mitigated	Same-language	10
M2	Mitigated	Same-language	10
M3	Mitigated	Same-language	2

Table 7. Galápagos' miscompilation mitigation results.

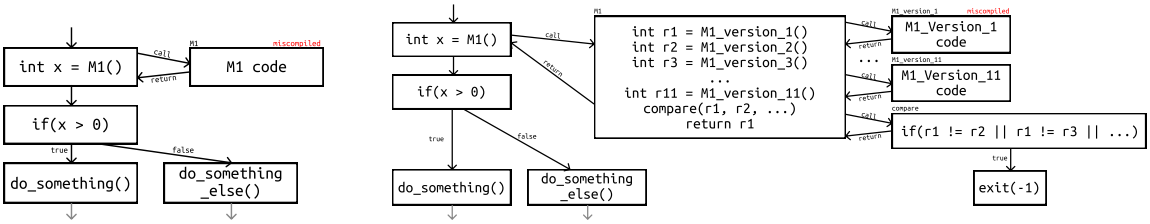


Fig. 11. Control flow graphs for M1. Left: single-version implementation. Right: N-Version implementation.

```
...
0x116b <n_version_call+14>      call   0x14d7 <version_1>            ;returns 0xffffffff
0x1170 <n_version_call+19>      mov    %eax,%ebx
0x1172 <n_version_call+21>      xor    %edi,%edi
0x1174 <n_version_call+23>      xor    %esi,%esi
0x1176 <n_version_call+25>      call   0x1240 <version_2>            ;returns 0x00000000
0x117b <n_version_call+30>      mov    %eax,0x14(%rsp)
...

0x11ea <n_version_call+141>     cmp    0x14(%rsp),%ebx               ;comparing <version_1> vs <version_2>
0x11ee <n_version_call+145>     jne    0x122f <n_version_call+210>
...
0x122f <n_version_call+210>     ud2                                  ;crash!
...
```

Fig. 12. Disassembled machine code for the hardened M2 function.

Upon manual inspection of the execution of the resulting binaries, we find that none of the equivalent variants are affected by the miscompilation bugs, only the original function is. As a result, the program termination is triggered by the inconsistent return values between the original function and the return values of one of the corresponding variants.

Figure 12 shows an excerpt of the machine code produced by Galápagos, when attempting mitigation of M2. The figure shows the calls to the first two versions of the resulting 11-Version implementation: `<version_1>` is the original function, which is miscompiled; and `<version_2>` is one of the variants proven to be equivalent at the IR level. In this example, `<version_1>` returns `-1`, while `<version_2>` returns `0`. Later in the execution, the return values are compared, and since these are different, the execution is redirected to the `ud2` instruction, triggering a crash.

Answer to RQ3

Galápagos is able to generate N-Version programs that mitigate the considered real-world miscompilation bugs. Our results show that equivalent function variants produced by LLMs can be employed to harden critical sections of a given program by assembling N-Version functions.

5.5 Performance Overhead

Table 8 shows a summary of the overhead introduced by the N-Version functions when compared to the reference functions. The column "Baseline Runtime" shows the measurement taken for each function as described in subsection 4.6. For both same-language and cross-language configurations, "N" represents the number of versions used in the maximally diverse N-Version function; "N-Version runtime" shows the execution time for the N-Version function; and finally "Normalized Overhead" shows the average overhead for a N-Version function with one single-variant (i.e. a 2-version function).

Function Name	Baseline runtime (μs)	Same-language diversification (C)			Cross-language diversification (C+Go)		
		N	N-Version runtime (μs)	Normalized Overhead	N	N-Version runtime (μs)	Normalized Overhead
alaw_to_s16	4 606	2	17 256	3.75x	7	38 363	2.22x
iec958_parity	83 623	5	269 676	1.56x	5	95 393	1.04x
add	2 486	10	36 362	2.51x	2	5 879	2.36x
ulaw_to_s16	4 369	10	54 707	2.28x	11	52 961	2.11x
val_seg	4 163	3	12 544	2.01x	–	–	–
flac_get_max_frame_size	6 222	5	30 622	1.98x	–	–	–
mix	9 067	7	60 226	1.94x	–	–	–
weight	6 256	11	59 428	1.85x	–	–	–
icbrt64	64 412	8	700 098	2.41x	–	–	–
BitDeinterleave	1 745	–	–	–	4	7 309	2.06x
BitInterleave	1 848	–	–	–	2	4 293	2.32x
_booth_recode_w5	4 599	3	13 757	2.0x	3	11 070	1.7x
fpr_half	2 895	3	12 139	2.6x	2	5 833	2.01x
fpr_lt	5 615	–	–	–	2	13 796	2.46x
modp_montymul	5 543	3	20 754	2.37x	–	–	–
modp_norm	2 793	–	–	–	3	8 210	1.97x
int16_nonzero_mask	3 872	4	15 601	2.01x	9	25 391	1.69x
barrett_reduce	3 095	–	–	–	11	67 948	3.1x
ctz	212 299	10	866 981	1.34x	7	655 523	1.35x
int16_t_negative_mask	2 754	8	25 538	2.18x	6	16 344	1.99x
int16_t_nonzero_mask	4 060	8	27 467	1.82x	6	18 922	1.73x
montgomery_reduce	2 472	5	18 241	2.59x	9	17 304	1.75x
fBlaMka	3 356	10	38 936	2.18x	–	–	–
Avg. Overhead				2.18x			1.99x

Table 8. Runtime measurement of N-Version functions.

The average normalized overhead is expected to be 2x for a 2-version function, which is what we observe on the last average row. The observed maximum and minimum overheads are 3.75x and 1.04x respectively. The minimum value can be considered an outlier, and it is introduced by a loop-unroll compiler optimization in the Go variant of a C function. We note that it is also possible to design parallel execution harnesses in time-critical contexts.

Answer to RQ4

Our experiments show that the N-Version functions generated by Galápagos introduce the expected overhead. After executing the N-Version functions, we observe that the runtime is increased in average by 2x for a 2-version function, and grows linearly by adding more variants to the assembly. This experiment provides evidence that Galápagos's harness pass' overhead is negligible. We note that variants may introduce algorithmic differences in the diversified function (e.g. iteration vs recursion) changing the diversified function complexity itself, yet this does not change the overhead due to the N-Version assembly itself.

6 Discussion

In this section, we discuss Galápagos in the larger context of software development, as well as the threats to the validity of our experiments.

6.1 Integration into Existing Software Development Practices

The integration of Galápagos into an existing software development project requires: (1) identifying critical sections of code with the highest reliability standards, and; (2) isolating them into pure functions. Developers would need to integrate Galápagos in the build pipeline for those functions. The pipeline would automatically generate variants, validate their correctness, and insert the N-Version functions into the final build. At deployment time, the N-Version functions would terminate execution in case of inconsistencies detected by diversification. This is valid in IoT [46]. We are also aware that companies providing security critical software for the military have such build pipelines with other kinds of diversification.

6.2 Threats to Validity

Threats to internal validity: Per our design, we rely on existing intermediate representations (LLVM IR) and equivalence checkers (alive-tv). Therefore, Galápagos will inherit their trustworthiness and limitations. For instance, in our experiments, some of the variants soundly discarded as non-equivalent were not proven as such, but `alive-tv` failed to process them because of unsupported LLVM instructions.

Also, the inherent non-determinism of LLMs can affect our results. This means that the proportion of equivalent variants can be different in an experimental reproduction. However, non-determinism is key in the design of Galápagos, as it is needed in the diversification pass to generate distinct variants.

Threats to external validity: We identify two dimensions where the obtained results can be generalized for Galápagos' design. First, our experiments focus on two language pairs: C-to-C and C-to-Go. We argue that similar results can be achieved for other language pairs, as long as these are IR-compatible, and a corresponding IR equivalence checking tool exists. Second, our experiments only consider a single LLM as a source of diversity. We argue that similar results can be achieved using different LLMs, as these have been shown to have similar capabilities [23]. We acknowledge that the generalizability of the results, and the claims of suitability of the design can be strengthened by performing experiments with different language pairs, IRs, generative models, and prompt templates.

7 Related Work

7.1 Automated Variant Synthesis

Singh et al. [47] propose the automatic creation of language-diverse, formally verified program variants from a high-level specification. They focus on fault-tolerance of distributed systems by executing each variant following the replicated state machine paradigm. However, no prototype implementation, experimental methodology, or results are provided. Pelofske et al. [23] present a thorough analysis of function variant generation with LLMs. This study is focused on four SHA-1 functions, and provides a deep insight into the diversity and correctness of LLM-generated variant functions. Xu et al. [22] describe a variant generation mechanism based on LLVM-IR obfuscation. Their goal is to create explicitly non-equivalent variants, which are resistant to replication of known tampering mechanisms. While these works aim at automating variant creation, and to different extents, correctness verification, the key novelty of our work is the focus on hardening of binaries via N-Version function execution.

7.2 LLMs code correctness

Pan et al. [48] identify the correctness of LLM-generated code as an important challenge to address. In this vein, Kessel and Atkinson [49] present LASSO, a platform for differential execution of large sets of code *versions* and tests. Compared to this work Galápagos goes one step further and automates the creation of N-Version programs, in addition to generation and testing of code variants. Du et al. [50] present a benchmark for solutions to programming tasks via LLMs. This benchmark is designed to measure how efficient are the generated solutions in terms of algorithmic complexity. The experiments show that LLMs are able to produce code that solves the tasks while having diverse efficiency scores. Zhang et al. [35] introduce a benchmark to quantify the correctness of code produced by an LLM. The experiments evidence that LLMs have the capability to correctly solve programming tasks to a large extent. However, while the data provided by these works gives a detailed overview of the correctness and efficiency of code produced by LLMs, the evaluations are test-based, and not formally verified against a specification.

7.3 Code Generation and Code Translation

Code generation and translation with LLMs are very active research areas [51, 52]. Yang et al. [27] present a C-to-Rust verified translation tool. It works by using static methods to construct a known equivalent translation, and LLMs to create more idiomatic translations. The latter are then checked for equivalence against the former using an off-the-shelf Rust tool. Bathia et al. [53] describe a tool to produce verified code transpilations. This is achieved by leveraging intermediate representations and formal equivalence checkers. Yang et al. [54] present UniTrans, a framework to generate and test LLM code translation. UniTrans uses the original code to further generate test cases for the resulting translation, and leverage them to iteratively repair the translation in case of failure. While the mentioned works address the issue of validating correctness of LLM-generated code equivalence, their scope is not concerned with software reliability.

7.4 Code Transplantation and Recombination:

As part of Galápagos' Harnessing pass, IR code from newly generated programs is inserted into the project that is being hardened. This process is similar to the code transplantation mechanisms described in the literature. This process has been studied in the context of automated program repair [55], feature transfer between systems [56–58], and code clone analysis [59]. The Harnessing pass is also similar to recombination approaches, where different versions of systems are created by manipulating sections of readily available codebases, at lower-than-source levels [60, 61]. While closely related, these works are not concerned with the challenges of the two first passes of Galápagos, namely, automatic diversification and correctness checking.

7.5 Addressing Miscompilations

The works of Eide et al. [31] and Le et al. [62] present approaches to discover miscompilation bugs via stochastic exploration of the program space. With the same goal, Tu et al. [63] describe a mechanism with guided LLM prompt production approach, which allows producing bug-triggering program mutations. While these approaches seek to actively discover miscompilation bugs, Galápagos takes a program hardening, as it does not target compilers directly, but protects critical application code against potential miscompilation.

8 Future Work

Looking ahead, we see concrete directions opened by this research.

8.1 Independence of LLM-Generated Variants

A fundamental challenge in N-Version programming is ensuring that different implementations fail independently under the same inputs. While our current approach generates variants that are statically and dynamically different, we need to explore how to measure independence of implementations. This includes defining quantitative metrics for independence, establishing experimental protocols to estimate fault correlation under diverse inputs. Then, one could use these metrics to synthesize variants that actively maximize independence. Future work should also investigate whether LLM-generated variants exhibit different failure patterns, not only between themselves, but compared to human-written implementations.

8.2 LLM Prompting and Search Approaches

The current diversification approach uses a single-shot prompting strategy. Future work should explore more sophisticated approaches to variant generation. This includes casting variant synthesis as constrained search that maximizes behavioral diversity under an equivalence oracle, and exploiting verifier counterexamples to repair failing variants. We should also investigate measurement-driven synthesis, where prompts are conditioned on diversity metrics (e.g., edit distance, instruction-set Jaccard) to steer generation away from previously explored regions. Additionally, diversity-aware fine-tuning could adapt LLMs to the diversification task, with training objectives that encourage behavioral distance while preserving equivalence.

8.3 Language Support Extension

While our current implementation focuses on C and Go, extending Galápagos to support additional programming languages presents both opportunities and challenges. The primary technical barrier is the requirement for IR compatibility—languages must compile to a common intermediate representation (e.g., LLVM IR) to enable formal equivalence checking. This constraint limits support to languages with mature LLVM front-ends, such as Rust or Swift. The implementation cost for adding a new language pair involves: (1) ensuring robust compilation to LLVM IR, (2) adapting the validation pipeline to handle language-specific IR patterns, and (3) potentially developing language-specific equivalence checking tools if existing ones are insufficient. However, the modular design of Galápagos's three-pass architecture means that language support can be added incrementally without major architectural changes. The diversification pass is already language-agnostic, requiring only appropriate prompting strategies. The validation pass depends on IR-level tools that can be extended or replaced as needed. The harnessing pass operates at the IR level and is largely language-independent. Thus, while each new language pair requires careful engineering effort, the overall approach scales well to additional languages.

9 Conclusion

We introduced automated N-Version programming powered by LLMs and presented Galápagos, a practical pipeline that (1) diversifies functions at the source level, (2) validates candidates via compilation, testing, and formal IR-level equivalence checking, and (3) harnesses verified variants into executable N-Version software.

Across 30 real-world C functions, we found that LLMs can synthesize function variants that are provably equivalent, even across languages. We showed that a substantial portion of the induced diversity survives compiler optimizations and can be measured at runtime. We also demonstrated that assembling these variants as N-Version functions mitigates

real miscompilation bugs, by converting silent wrong results into explicit program halts. The added runtime cost matches expectations (~2x for a 2-version function) and scales linearly with N. Overall, our results indicate that combining practical formal methods with modern LLMs can substantially reduce the engineering cost of N-Version programming while giving significant correctness guarantees.

References

[1] Stephanie Forrest, Anil Somayaji, and David H Ackley. Building diverse computer systems. In *Proceedings. The Sixth Workshop on Hot Topics in Operating Systems (Cat. No. 97TB100133)*, pages 67–72. IEEE, 1997.

[2] Algirdas Avizienis. The n-version approach to fault-tolerant software. *IEEE Transactions on software engineering*, (12):1491–1501, 1985.

[3] Algirdas Avizienis. The methodology of n-version programming. *Software fault tolerance*, 3:23–46, 1995.

[4] John C Knight and Nancy G Leveson. An experimental evaluation of the assumption of independence in multiversion programming. *IEEE Transactions on software engineering*, (1):96–109, 2012.

[5] Susan S. Brilliant, John C. Knight, and Nancy G. Leveson. Analysis of faults in an n-version software experiment. *IEEE Transactions on software engineering*, 16(2):238–247, 2002.

[6] Lui Sha. Using simplicity to control complexity. *IEEE Software*, 18(4):20–28, 2001.

[7] Mark Chen et al. Evaluating large language models trained on code, 2021.

[8] Dávid Hidvégi, Khashayar Etemadi, Sofia Bobadilla, and Martin Monperrus. Cigar: Cost-efficient program repair with llms, 2024.

[9] Toufique Ahmed and Premkumar Devanbu. Few-shot training llms for project-specific code-summarization. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, pages 1–5, 2022.

[10] The tigress C obfuscator. tigress.wtf.

[11] Javier Cabrera-Arteaga, Nicholas Fitzgerald, Martin Monperrus, and Benoit Baudry. Wasm-mutate: Fast and effective binary diversification for webassembly. *Computers & Security*, 139:103731, 2024.

[12] Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. Finding and understanding bugs in c compilers. In *Proceedings of the 32nd ACM SIGPLAN conference on Programming language design and implementation*, pages 283–294, 2011.

[13] D.E. Eckhardt and L.D. Lee. A theoretical basis for the analysis of multiversion software subject to coincident errors. *IEEE Transactions on Software Engineering*, SE-11(12):1511–1517, 1985.

[14] Javier Ron, César Soto-Valero, Long Zhang, Benoit Baudry, and Martin Monperrus. Highly available blockchain nodes with n-version design. *IEEE Transactions on Dependable and Secure Computing*, 2023.

[15] K.S. Tso, A. Avižienis, and J.P.J. Kelly. Error recovery in multi-version software. *IFAC Proceedings Volumes*, 19(11):35–41, 1986. 5th IFAC Workshop on Safety of Computer Control Systems 1986 (SAFECOMP '86). Trends in Safe Real Time Computer Systems, Sarlat, France, 14-17 October, 1986.

[16] Javier Ron, Zheyuan He, and Martin Monperrus. Proving and rewarding client diversity to strengthen resilience of blockchain networks. In *ACM Distributed Ledger Technologies: Research and Practice*, 2025.

[17] Sara Gholami, Alireza Goli, Cor-Paul Bezemer, and Hamzeh Khazaei. A framework for satisfying the performance requirements of containerized software systems through multi-versioning. In *Proceedings of the ACM/SPEC International Conference on Performance Engineering*, pages 150–160, 2020.

[18] Meng Xu, Kangjie Lu, Taesoo Kim, and Wenke Lee. Bunshin: compositing security mechanisms through diversification. In *2017 USENIX Annual Technical Conference (USENIX ATC 17)*, pages 271–283, 2017.

[19] Antonio M. Espinoza, Riley Wood, Stephanie Forrest, and Mohit Tiwari. Back to the future: N-versioning of microservices. In *52nd Annual IEEE/IFIP International Conference on Dependable Systems and Networks, DSN 2022, Baltimore, MD, USA, June 27-30, 2022*, pages 415–427. IEEE, 2022.

[20] Tingting Hu, Ivan Cibrario Bertolott, and Nicolas Navet. Towards seamless integration of n-version programming in model-based design. In *2017 22nd IEEE International Conference on Emerging Technologies and Factory Automation (ETFA)*, pages 1–8, 2017.

[21] Petr Hosek and Cristian Cadar. Varan the unbelievable: An efficient n-version execution framework. *SIGARCH Comput. Archit. News*, 43(1):339–353, mar 2015.

[22] Hui Xu, Yangfan Zhou, and Michael Lyu. N-version obfuscation. In *Proceedings of the 2nd ACM International Workshop on Cyber-Physical System Security*, CPSS '16, page 22–33, New York, NY, USA, 2016. Association for Computing Machinery.

[23] Elijah Pelofske, Vincent Urias, and Lorie M. Liebrock. Automated creation of source code variants of a cryptographic hash function implementation using generative pre-trained transformer models, 2024.

[24] Michele Tufano, Jevgenija Pantiuchina, Cody Watson, Gabriele Bavota, and Denys Poshyvanyk. On learning meaningful code changes via neural machine translation. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*, pages 25–36, 2019.

[25] Zimin Chen, Steve Kommrusch, Michele Tufano, Louis-Noël Pouchet, Denys Poshyvanyk, and Martin Monperrus. Sequencer: Sequence-to-sequence learning for end-to-end program repair. *IEEE Transactions on Software Engineering*, 47(9):1943–1959, 2021.

[26] Marie-Anne Lachaux, Baptiste Roziere, Lowik Chanussot, and Guillaume Lample. Unsupervised translation of programming languages, 2020.

[27] Aidan Z. H. Yang, Yoshiki Takashima, Brandon Paulsen, Josiah Dodds, and Daniel Kroening. Vert: Verified equivalent rust transpilation with large language models as few-shot learners, 2024.

[28] Chengnian Sun, Vu Le, Qirun Zhang, and Zhendong Su. Toward understanding compiler bugs in gcc and llvm. In *Proceedings of the 25th international symposium on software testing and analysis*, pages 294–305, 2016.

[29] Xavier Leroy. The compcert c verified compiler: Documentation and user's manual. Technical report, Inria, 2023.

[30] Michaël Marcozzi, Qiyi Tang, Alastair F Donaldson, and Cristian Cadar. Compiler fuzzing: How much does it matter? *Proceedings of the ACM on Programming Languages*, 3(OOPSLA):1–29, 2019.

[31] Eric Eide and John Regehr. Volatiles are miscompiled, and what to do about it. In *Proceedings of the 8th ACM International Conference on Embedded Software*, EMSOFT '08, page 255–264, New York, NY, USA, 2008. Association for Computing Machinery.

[32] Scott Blyth, Christoph Treude, and Markus Wagner. Creative and correct: Requesting diverse code solutions from ai foundation models. In *Proceedings of the 2024 IEEE/ACM First International Conference on AI Foundation Models and Software Engineering*, FORGE '24, page 119–123, New York, NY, USA, 2024. Association for Computing Machinery.

[33] Andrei Homescu, Steven Neisius, Per Larsen, Stefan Brunthaler, and Michael Franz. Profile-guided automated software diversity. In *Proceedings of the 2013 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, pages 1–11, 2013.

[34] Vasileios P. Kemerlis, Georgios Portokalidis, and Angelos D. Keromytis. kguard: lightweight kernel protection against return-to-user attacks. In *Proceedings of the 21st USENIX Conference on Security Symposium*, Security'12, page 39, USA, 2012. USENIX Association.

[35] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and LINGMING ZHANG. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. In A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 21558–21572. Curran Associates, Inc., 2023.

[36] Fang Liu, Yang Liu, Lin Shi, Houkun Huang, Ruifeng Wang, Zhen Yang, and Li Zhang. Exploring and evaluating hallucinations in llm-powered code generation. *arXiv preprint arXiv:2404.00971*, 2024.

[37] Nuno P Lopes, Juneyoung Lee, Chung-Kil Hur, Zhengyang Liu, and John Regehr. Alive2: bounded translation validation for llvm. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, pages 65–79, 2021.

[38] Alexa VanHattum, Daniel Schwartz-Narbonne, Nathan Chong, and Adrian Sampson. Verifying dynamic trait objects in rust. In *Proceedings of the 44th International Conference on Software Engineering: Software Engineering in Practice*, ICSE-SEIP '22, page 321–330, New York, NY, USA, 2022. Association for Computing Machinery.

[39] Lorenz Breidenbach, Phil Daian, Florian Tramèr, and Ari Juels. Enter the hydra: Towards principled bug bounties and Exploit-Resistant smart contracts. In *27th USENIX Security Symposium (USENIX Security 18)*, pages 1335–1352, Baltimore, MD, August 2018. USENIX Association.

[40] Wei Huang, Ana L. Milanova, Werner Dietl, and Michael D. Ernst. Reim & reiminfer: checking and inference of reference immutability and method purity. In Gary T. Leavens and Matthew B. Dwyer, editors, *Proceedings of the 27th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications, OOPSLA 2012, part of SPLASH 2012, Tucson, AZ, USA, October 21-25, 2012*, pages 879–896. ACM, 2012.

[41] Jens Nicolay, Carlos Noguera, Coen De Roover, and Wolfgang De Meuter. Detecting function purity in javascript. In Michael W. Godfrey, David Lo, and Foutse Khomh, editors, *15th IEEE International Working Conference on Source Code Analysis and Manipulation, SCAM 2015, Bremen, Germany, September 27-28, 2015*, pages 101–110. IEEE Computer Society, 2015.

[42] Javier Cabrera Arteaga. *Software Diversification for WebAssembly*. PhD thesis, KTH Royal Institute of Technology, 2024.

[43] Chi-Keung Luk, Robert Cohn, Robert Muth, Harish Patil, Artur Klauser, Geoff Lowney, Steven Wallace, Vijay Janapa Reddi, and Kim Hazelwood. Pin: building customized program analysis tools with dynamic instrumentation. In *Proceedings of the 2005 ACM SIGPLAN Conference on Programming Language Design and Implementation*, PLDI '05, page 190–200, New York, NY, USA, 2005. Association for Computing Machinery.

[44] Michael Levandowsky and David Winter. Distance between sets. *Nature*, 234(5323):34–35, 1971.

[45] Christian Collberg, Sam Martin, Jonathan Myers, and Jasvir Nagra. Distributed application tamper detection via continuous software updates. In *Proceedings of the 28th Annual Computer Security Applications Conference*, pages 319–328, 2012.

[46] Hussain Almohri, Layne Watson, David Evans, and Stephen Billups. Dynamic system diversification for securing cloud-based iot subnetworks. *ACM Transactions on Autonomous and Adaptive Systems (TAAS)*, 17(1-2):1–23, 2022.

[47] Atul Singh, Nishant Sinha, and Nitin Agrawal. Avatars for pennies: Cheap n-version programming for replication. In *6th Workshop on Hot Topics in System Dependability*, 2010.

[48] Rangeet Pan, Ali Reza Ibrahimzada, Rahul Krishna, Divya Sankar, Lambert Pouguem Wassi, Michele Merler, Boris Sobolev, Raju Pavuluri, Saurabh Sinha, and Reyhaneh Jabbarvand. Lost in translation: A study of bugs introduced by large language models while translating code. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, ICSE '24, New York, NY, USA, 2024. Association for Computing Machinery.

[49] Marcus Kessel and Colin Atkinson. N-version assessment and enhancement of generative ai. *IEEE Software*, page 1–8, 2024.

[50] Mingzhe Du, Anh Tuan Luu, Bin Ji, Qian Liu, and See-Kiong Ng. Mercury: A code efficiency benchmark for code large language models, 2024.

[51] Zibin Zheng, Kaiwen Ning, Yanlin Wang, Jingwen Zhang, Dewu Zheng, Mingxi Ye, and Jiachi Chen. A survey of large language models for code: Evolution, benchmarking, and future trends, 2024.

[52] Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. A survey on large language models for code generation, 2024.

[53] Sahil Bhatia, Jie Qiu, Niranjan Hasabnis, Sanjit A. Seshia, and Alvin Cheung. Verified code transpilation with llms, 2024.

[54] Zhen Yang, Fang Liu, Zhongxing Yu, Jacky Wai Keung, Jia Li, Shuo Liu, Yifan Hong, Xiaoxue Ma, Zhi Jin, and Ge Li. Exploring and unleashing the power of large language models in automated code translation. *Proc. ACM Softw. Eng.*, 1(FSE), July 2024.

[55] Sergey Mechtaev, Manh-Dung Nguyen, Yannic Noller, Lars Grunske, and Abhik Roychoudhury. Semantic program repair using a reference implementation. In *Proceedings of the 40th International Conference on Software Engineering*, pages 129–139, 2018.

[56] Stelios Sidiroglou-Douskos, Eric Lahtinen, Anthony Eden, Fan Long, and Martin Rinard. Codecarboncopy. In *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering*, pages 95–105, 2017.

[57] Earl T. Barr, Yuriy Brun, Premkumar Devanbu, Mark Harman, and Federica Sarro. The plastic surgery hypothesis. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*, FSE 2014, page 306–317, New York, NY, USA, 2014. Association for Computing Machinery.

[58] Mark Harman, Yue Jia, and William B Langdon. Babel pidgin: Sbse can grow and graft entirely new functionality into a real world system. In *Search-Based Software Engineering: 6th International Symposium, SSBSE 2014, Fortaleza, Brazil, August 26-29, 2014. Proceedings 6*, pages 247–252. Springer, 2014.

[59] Tianyi Zhang and Miryung Kim. Automated transplantation and differential testing for clones. In *2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE)*, pages 665–676, 2017.

[60] Bheesham Persaud, Borke Obada-Obieh, Nilofar Mansourzadeh, Ashley Moni, and Anil Somayaji. Frankenssl: Recombining cryptographic libraries for software diversity. In *Proceedings of the 11th Annual Symposium On Information Assurance. NYS Cyber Security Conference*, pages 19–25, 2016.

[61] Blair Foster and Anil Somayaji. Object-level recombination of commodity applications. In *Proceedings of the 12th Annual Conference on Genetic and Evolutionary Computation*, GECCO '10, page 957–964, New York, NY, USA, 2010. Association for Computing Machinery.

[62] Vu Le, Chengnian Sun, and Zhendong Su. Finding deep compiler bugs via guided stochastic program mutation. *SIGPLAN Not.*, 50(10):386–399, oct 2015.

[63] Haoxin Tu, Zhide Zhou, He Jiang, Imam Nur Bani Yusuf, Yuxian Li, and Lingxiao Jiang. Isolating compiler bugs by generating effective witness programs with large language models. *IEEE Transactions on Software Engineering*, 50(7):1768–1788, 2024.